

GCP Hands-on Lab: Compute Engine, Cloud SQL, Networking, IAM, Ansible PostgreSQL

You will create a VPC + subnet, firewall, service accounts, one Compute Engine VM (Ansible installs PostgreSQL 16 here), and one Cloud SQL for PostgreSQL instance (private IP). This is separate from the Zalando/Patroni Kubernetes cluster.

Do not attach a public IP to Cloud SQL. Do not open 5432 to 0.0.0.0/0. Delete VM, instance, and disks at the end so the project is not billed overnight.

0. Variables (set once)

```
export PROJECT_ID="$(gcloud config get-value project)"
export REGION=asia-south1
export ZONE=asia-south1-a
export NETWORK=pg-lab-vpc
export SUBNET=pg-lab-subnet
export SUBNET_RANGE=10.20.0.0/24
export VM=pg-ansible-vm
export SA_VM=pg-vm-sa
export SA_SQL=pg-sql-client-sa
export SQL_INSTANCE=pg-lab-sql
export SQL_TIER=db-custom-1-3840
echo "PROJECT_ID=$PROJECT_ID"
gcloud services enable compute.googleapis.com sqladmin.googleapis.com iam.googleapis.com servicenetworking.googleapis.com
```

1. Configure networking

1.1 VPC and subnet

```
gcloud compute networks create "$NETWORK" --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets create "$SUBNET" \
  --network="$NETWORK" --region="$REGION" --range="$SUBNET_RANGE" --enable-private-ip-google-access
```

1.2 Firewall — IAP SSH only + Postgres from subnet

```
gcloud compute firewall-rules create pg-lab-allow-iap-ssh \
  --network="$NETWORK" --allow=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=pg-lab

gcloud compute firewall-rules create pg-lab-allow-5432-internal \
  --network="$NETWORK" --allow=tcp:5432 --source-ranges="$SUBNET_RANGE" --target-tags=pg-lab
```

35.235.240.0/20 is Identity-Aware Proxy for SSH. Do not add 0.0.0.0/0 on 22 or 5432.

1.3 Private services access for Cloud SQL

```
gcloud compute addresses create pg-lab-psa --global --purpose=VPC_PEERING \
  --prefix-length=16 --network="$NETWORK"
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com \
  --ranges=pg-lab-psa --network="$NETWORK"
```

2. Configure IAM

```
gcloud iam service-accounts create "$SA_VM" --display-name="PostgreSQL VM"
gcloud iam service-accounts create "$SA_SQL" --display-name="Cloud SQL client"

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_VM}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/logging.logwriter"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_VM}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/monitoring.metricwriter"

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_SQL}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/cloudsql.client"
```

VM SA: logs/metrics only. SQL client SA: connect via Cloud SQL Auth Proxy if you add it later. Least privilege — not Editor.

3. Deploy Compute Engine

```
gcloud compute instances create "$VM" \
  --zone="$ZONE" \
  --machine-type=e2-medium \
  --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
  --boot-disk-size=20GB --boot-disk-type=pd-balanced \
```

```
--network-interface=network="$NETWORK",subnet="$SUBNET",no-address \
--tags=pg-lab \
--service-account="${SA_VM}@${PROJECT_ID}.iam.gserviceaccount.com" \
--scopes=https://www.googleapis.com/auth/cloud-platform \
--metadata=enable-oslogin=TRUE
```

```
gcloud compute instances describe "$VM" --zone="$ZONE" --format='get(networkInterfaces[0].networkIP,status)'
```

no-address = no public IP. SSH with IAP:

```
gcloud compute ssh "$VM" --zone="$ZONE" --tunnel-through-iap
```

4. Provision Cloud SQL

```
gcloud sql instances create "$SQL_INSTANCE" \
--database-version=POSTGRES_16 \
--tier="$SQL_TIER" \
--region="$REGION" \
--network="projects/${PROJECT_ID}/global/networks/${NETWORK}" \
--no-assign-ip \
--availability-type=zonal \
--storage-size=20 --storage-type=SSD \
--root-password="ChangeMe-Lab-Only"
```

```
gcloud sql databases create appdb --instance="$SQL_INSTANCE"
gcloud sql users create appuser --instance="$SQL_INSTANCE" --password="ChangeMe-App-Only"
gcloud sql instances describe "$SQL_INSTANCE" --format='get(ipAddresses)'
```

Private IP only. From the VM, install the Auth Proxy or whitelist the VM IP in authorized networks if the instructor allows. Default: use Auth Proxy.

```
# on the VM (after SSH)
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.3/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy
./cloud-sql-proxy --private-ip ${PROJECT_ID}:${REGION}:${SQL_INSTANCE} --port=5433
# other terminal:
sudo apt-get update && sudo apt-get install -y postgresql-client
psql "host=127.0.0.1 port=5433 user=appuser dbname=appdb" -c "SELECT current_user, inet_server_addr();"

```

5. Install PostgreSQL on the VM with Ansible

5.1 Control node packages (your laptop or the VM itself for a single-host lab)

```
sudo apt-get update
sudo apt-get install -y ansible python3-pip
ansible-galaxy collection install community.postgresql community.general
```

5.2 Inventory (IAP / private IP). For a single-host install, use localhost.

```
mkdir -p ~/pg-ansible/{inventories,playbooks,templates}
cat > ~/pg-ansible/inventories/lab.ini <<'EOF'
[postgres]
pg-ansible-vm ansible_host=10.20.0.2 ansible_user=YOUR_USER ansible_python_interpreter=/usr/bin/python3

[postgres:vars]
pg_version=16
pg_listen=*
pg_allowed_cidr=10.20.0.0/24
EOF
# replace 10.20.0.2 with the VM internal IP from step 3
```

5.3 Playbook — install, configure, user, backup dir

```
cat > ~/pg-ansible/playbooks/postgres.yml <<'EOF'
---
- name: Install and configure PostgreSQL
  hosts: postgres
  become: true
  vars:
    pg_version: 16
  tasks:
    - name: Packages
      apt:
        name:
          - "postgresql-{{ pg_version }}"
          - "postgresql-contrib-{{ pg_version }}"
          - python3-psycopg2
        update_cache: true
        state: present

    - name: Listen addresses
```

```

lineinfile:
  path: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf"
  regexp: "^#?listen_addresses"
  line: "listen_addresses = '*'"
notify: reload postgres

- name: pg_hba subnet
  lineinfile:
    path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
    line: "host all all 10.20.0.0/24 scram-sha-256"
  notify: reload postgres

- name: Backup directory
  file:
    path: /var/backups/postgresql
    state: directory
    owner: postgres
    group: postgres
    mode: "0750"

- name: App role
  become_user: postgres
  community.postgresql.postgresql_user:
    name: appuser
    password: "ChangeMe-App-Only"
    role_attr_flags: LOGIN

- name: App database
  become_user: postgres
  community.postgresql.postgresql_db:
    name: appdb
    owner: appuser

handlers:
  - name: reload postgres
    service:
      name: postgresql
      state: reloaded
EOF

# single-host (run ON the VM)
cd ~/pg-ansible
ansible-playbook playbooks/postgres.yml -i 'localhost,' -c local

# or from a control node with IAP SSH
# ansible-playbook -i inventories/lab.ini playbooks/postgres.yml

```

5.4 Verify on-VM PostgreSQL

```

sudo systemctl status postgresql --no-pager
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres psql -c "\du"
sudo -u postgres psql -d appdb -c "CREATE TABLE IF NOT EXISTS lab(id int); INSERT INTO lab VALUES (1); SELECT * FROM lab;"
sudo -u postgres pg_dump -Fc appdb -f /var/backups/postgresql/appdb.dump
ls -l /var/backups/postgresql/

```

6. What you built

Piece	Purpose
pg-lab-vpc / subnet	Private 10.20.0.0/24 + Private Google Access
Firewall IAP + 5432	SSH via IAP; Postgres only from subnet
PSA peering	Cloud SQL private IP
pg-vm-sa / pg-sql-client-sa	Least-privilege identities
pg-ansible-vm	Self-managed PostgreSQL via Ansible
pg-lab-sql	Managed Cloud SQL Postgres 16, no public IP

7. Cleanup (required)

```

gcloud sql instances delete "$SQL_INSTANCE" --quiet
gcloud compute instances delete "$VM" --zone="$ZONE" --quiet
gcloud compute firewall-rules delete pg-lab-allow-iap-ssh pg-lab-allow-5432-internal --quiet
gcloud services vpc-peerings delete --service=servicenetworking.googleapis.com --network="$NETWORK" --quiet || true
gcloud compute addresses delete pg-lab-psa --global --quiet || true
gcloud compute networks subnets delete "$SUBNET" --region="$REGION" --quiet
gcloud compute networks delete "$NETWORK" --quiet

```

```
# optionally delete SAs
gcloud iam service-accounts delete "${SA_VM}@${PROJECT_ID}.iam.gserviceaccount.com" --quiet
gcloud iam service-accounts delete "${SA_SQL}@${PROJECT_ID}.iam.gserviceaccount.com" --quiet
```

Pass criteria

VPC/subnet/firewall exist; VM has no public IP and is reachable with IAP SSH; Cloud SQL has only a private IP and appdb/appuser; Ansible installed PostgreSQL on the VM; pg_dump wrote /var/backups/postgresql/appdb.dump; resources deleted or scheduled for delete.